Assignment-15.3

2303A51304

Batch-05

Task-01:

Prompt:

Create a fully working REST API for an Online Food Ordering System using Node.js and Express with in-memory storage.
Implement endpoints:
GET /menu (list dishes with pagination)
POST /order (place order)
GET /order/{id} (track order status)
PUT /order/{id} (update order items)
DELETE /order/{id} (cancel order)
Include unique order IDs, order status flow, input validation, token-based authentication middleware, proper HTTP responses, error handling, sample data, clean folder structure, and instructions to run locally.

Code:

```python
# The API includes endpoints for managing the menu and orders, with basic authentication and error handling.
from flask import Flask, request, jsonify
import uuid

app = Flask(__name__)

# ----------------------------
# SAMPLE MENU DATA
# ----------------------------
menu = [
    {"id": 1, "name": "Pizza", "price": 10.99, "availability": True},
    {"id": 2, "name": "Burger", "price": 8.99, "availability": True},
    {"id": 3, "name": "Pasta", "price": 12.99, "availability": False},
    {"id": 4, "name": "Sandwich", "price": 6.99, "availability": True},
    {"id": 5, "name": "Fries", "price": 4.99, "availability": True}
]

orders = {}

VALID_TOKEN = "valid_token"

ORDER_STATUSES = [
    "Placed",
    "Preparing",
    "Out for Delivery",
    "Delivered",
    "Cancelled"
]

# ----------------------------
# AUTHENTICATION MIDDLEWARE
# ----------------------------
@app.before_request
def authenticate():
    token = request.headers.get("Authorization")
    if token != VALID_TOKEN:
        return jsonify({"error": "Unauthorized"}), 401
```

```python
# AUTHENTICATION MIDDLEWARE
# ----------------------------
@app.before_request
def authenticate():
    token = request.headers.get("Authorization")
    if token != VALID_TOKEN:
        return jsonify({"error": "Unauthorized"}), 401

# ----------------------------
# ERROR HANDLER
# ----------------------------
@app.errorhandler(Exception)
def handle_error(e):
    return jsonify({"error": str(e)}), 500

# ----------------------------
# GET /menu (Pagination)
# ----------------------------
@app.route("/menu", methods=["GET"])
def get_menu():
    page = int(request.args.get("page", 1))
    limit = int(request.args.get("limit", 2))

    start = (page - 1) * limit
    end = start + limit

    return jsonify({
        "page": page,
        "data": menu[start:end],
        "total_items": len(menu)
    }), 200


# ----------------------------
# POST /order (Create Order)
# ----------------------------
@app.route("/order", methods=["POST"])
```

Ln 79, Col 55 Spaces: 4

```python
# ----------------------------
# POST /order (Create Order)
# ----------------------------
@app.route("/order", methods=["POST"])
def create_order():
    data = request.get_json()

    if not data or "customer_name" not in data or "items" not in data:
        return jsonify({"error": "customer_name and items required"}), 400

    order_id = str(uuid.uuid4())

    orders[order_id] = {
        "order_id": order_id,
        "customer_name": data["customer_name"],
        "items": data["items"],
        "status": "Placed"
    }

    return jsonify(orders[order_id]), 201


# ----------------------------
# GET /order/{id}
# ----------------------------
@app.route("/order/<order_id>", methods=["GET"])
def get_order(order_id):
    order = orders.get(order_id)

    if not order:
        return jsonify({"error": "Order not found"}), 404

    return jsonify(order), 200


# ----------------------------
```

```python
# ----------------------------
# PUT /order/{id} (Update)
# ----------------------------
@app.route("/order/<order_id>", methods=["PUT"])
def update_order(order_id):
    order = orders.get(order_id)

    if not order:
        return jsonify({"error": "Order not found"}), 404

    data = request.get_json()

    if "items" in data:
        order["items"] = data["items"]

    if "status" in data:
        if data["status"] not in ORDER_STATUSES:
            return jsonify({"error": "Invalid status"}), 400
        order["status"] = data["status"]

    return jsonify(order), 200


# ----------------------------
# DELETE /order/{id}
# ----------------------------
@app.route("/order/<order_id>", methods=["DELETE"])
def cancel_order(order_id):
    order = orders.get(order_id)

    if not order:
        return jsonify({"error": "Order not found"}), 404

    order["status"] = "Cancelled"

    return jsonify({"message": "Order cancelled"}), 200


# ----------------------------
# RUN SERVER
# ----------------------------
if __name__ == "__main__":
    app.run(debug=True)
```

Output:

GET

Pretty | Raw | Preview

{ "data": [{ "availability": true, "id": 1, "name": "Pizza", "price": 10.99 }, { "availability": true, "id": 2, "name": "Burger", "price": 8.99 }], "page": 1, "total_items": 5 }

Post

{ "customer_name": "Poojitha", "items": ["Pizza", "Fries"], "order_id": "4a7816d5-0a0f-4da2-8d3b-ab7b6e7e0b52", "status": "Placed" }

Put

Pretty | Raw | Preview

{ "customer_name": "Poojitha", "items": ["Burger", "Fries"], "order_id": "baa66fbd-ea87-446c-99da-c3854b54e1a9", "status": "Preparing" }

Delete:

Pretty | Raw | Preview

{ "message": "Order cancelled" }

Explanation:

This Flask application builds a REST API for an online food ordering system that allows users to view a menu and manage orders. It stores menu data and orders in memory and generates unique order IDs using UUID. A token-based authentication middleware checks each request using an Authorization header before allowing access. The API provides endpoints to list menu items with pagination, create orders, track order details, update items or status, and cancel orders. It also validates inputs and returns proper JSON responses with appropriate HTTP status codes and error handling.

Task-02:

Prompt:

Create a RESTful Employee Payroll Management API using Python Flask.

Requirements:

- Use in-memory storage.

- Employee fields: id, name, position, salary.
- Endpoints:
  GET /employees
  POST /employees
  PUT /employees/<id>/salary
  DELETE /employees/<id>
- Add input validation.
- Use unique IDs.
- Add docstrings and comments.
- Return structured JSON responses with proper HTTP status codes.

Code:

ek-15.3 > task-02.py > add_employee

```python
- Add docstrings and comments.
- Return structured JSON responses with proper HTTP status codes.'''
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory database
employees = []
next_id = 1
@app.route('/employees', methods=['GET'])
def get_employees():
    """Return list of all employees."""
    return jsonify(employees), 200
@app.route('/employees', methods=['POST'])
def add_employee():
    """Add a new employee with salary."""
    global next_id
    data = request.json
    if not data.get("name") or not data.get("salary"):
        return jsonify({"error": "Name and salary required"}), 400

    employee = {
        "id": next_id,
        "name": data["name"],
        "position": data.get("position", "Employee"),
        "salary": data["salary"]
    }
    employees.append(employee)
    next_id += 1
    return jsonify(employee), 201
@app.route('/employees/<int:id>/salary', methods=['PUT'])
def update_salary(id):
    """Update employee salary."""
    data = request.json
    for emp in employees:
        if emp["id"] == id:
            emp["salary"] = data["salary"]
            return jsonify(emp), 200
```

```
k-15.3 > task-02.py > add_employee
def add_employee():
        "salary": data["salary"]
    }
    employees.append(employee)
    next_id += 1
    return jsonify(employee), 201
@app.route('/employees/<int:id>/salary', methods=['PUT'])
def update_salary(id):
    """Update employee salary."""
    data = request.json
    for emp in employees:
        if emp["id"] == id:
            emp["salary"] = data["salary"]
            return jsonify(emp), 200

    return jsonify({"error": "Employee not found"}), 404
@app.route('/employees/<int:id>', methods=['DELETE'])
def delete_employee(id):
    """Delete employee."""
    for emp in employees:
        if emp["id"] == id:
            employees.remove(emp)
            return jsonify({"message": "Employee removed"}), 200

    return jsonify({"error": "Employee not found"}), 404
if __name__ == '__main__':
    app.run(debug=True)
```

Output:

POST http://127.0.0.1:5000/employees

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
    "id":100,
    "name":"Poojitha",
    "salary":5000

}
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview

{ "id": 1, "name": "Poojitha", "position": "Employee", "salary": 5000 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

GET http://127.0.0.1:5000/employees

Params Authorization Headers (7) Body Scripts Settings

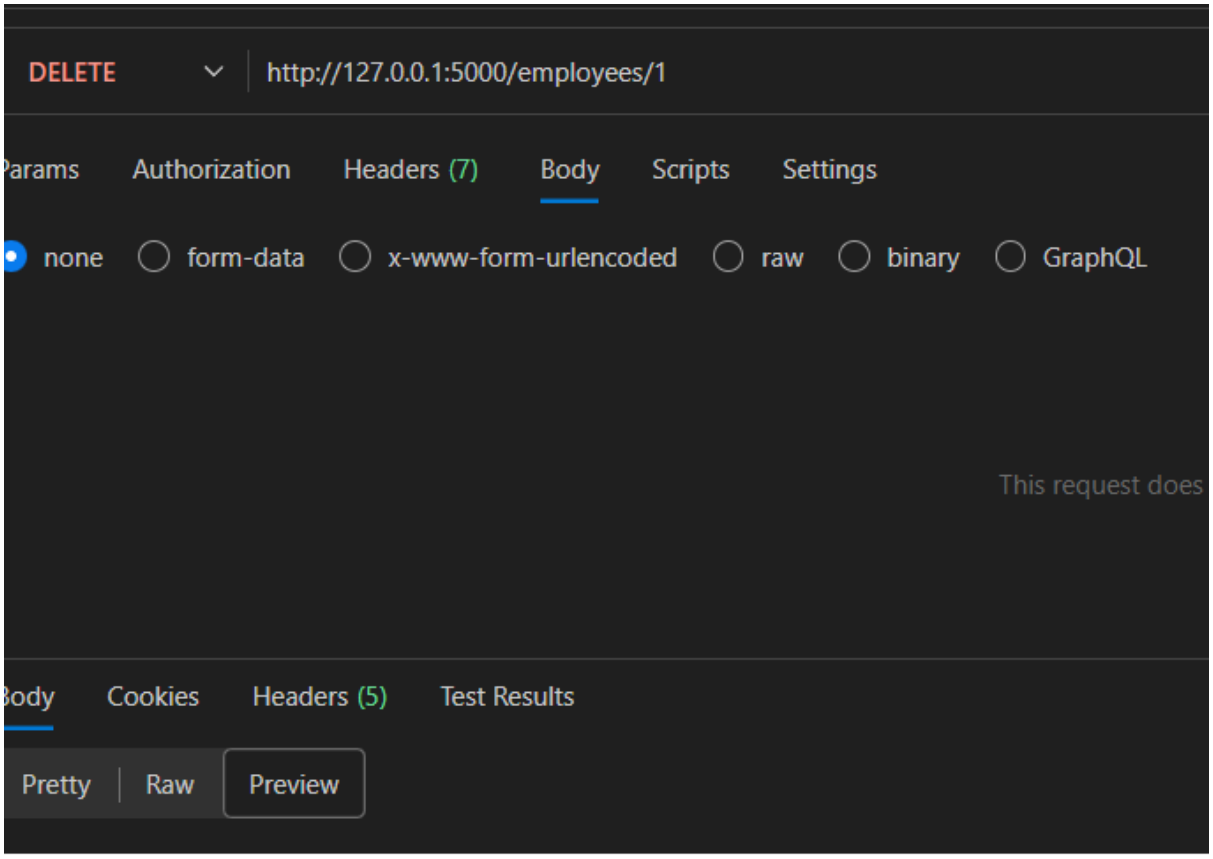
none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers (5) Test Results Status: 200 OK Time: 7 ms Size: 352

Pretty Raw Preview

[{ "id": 1, "name": "Poojitha", "position": "Employee", "salary": 5000 }, { "id": 2, "name": "Rahul", "position": "Employee", "salary": 7000 }]



{ "message": "Employee removed" }

Explanation:

This API manages employee records and salary details through REST endpoints.
It allows adding, viewing, updating salary, and deleting employees using structured requests.
An in-memory data model simulates payroll storage with unique employee IDs.
Validation and proper HTTP responses ensure secure and organized data handling.

Task-03:

Prompt:

Create a Student Management REST API using Flask.

Fields:

name, rollNumber, course, year

Endpoints:

GET /students

POST /students

PUT /students/<rollNumber>

DELETE /students/<rollNumber>

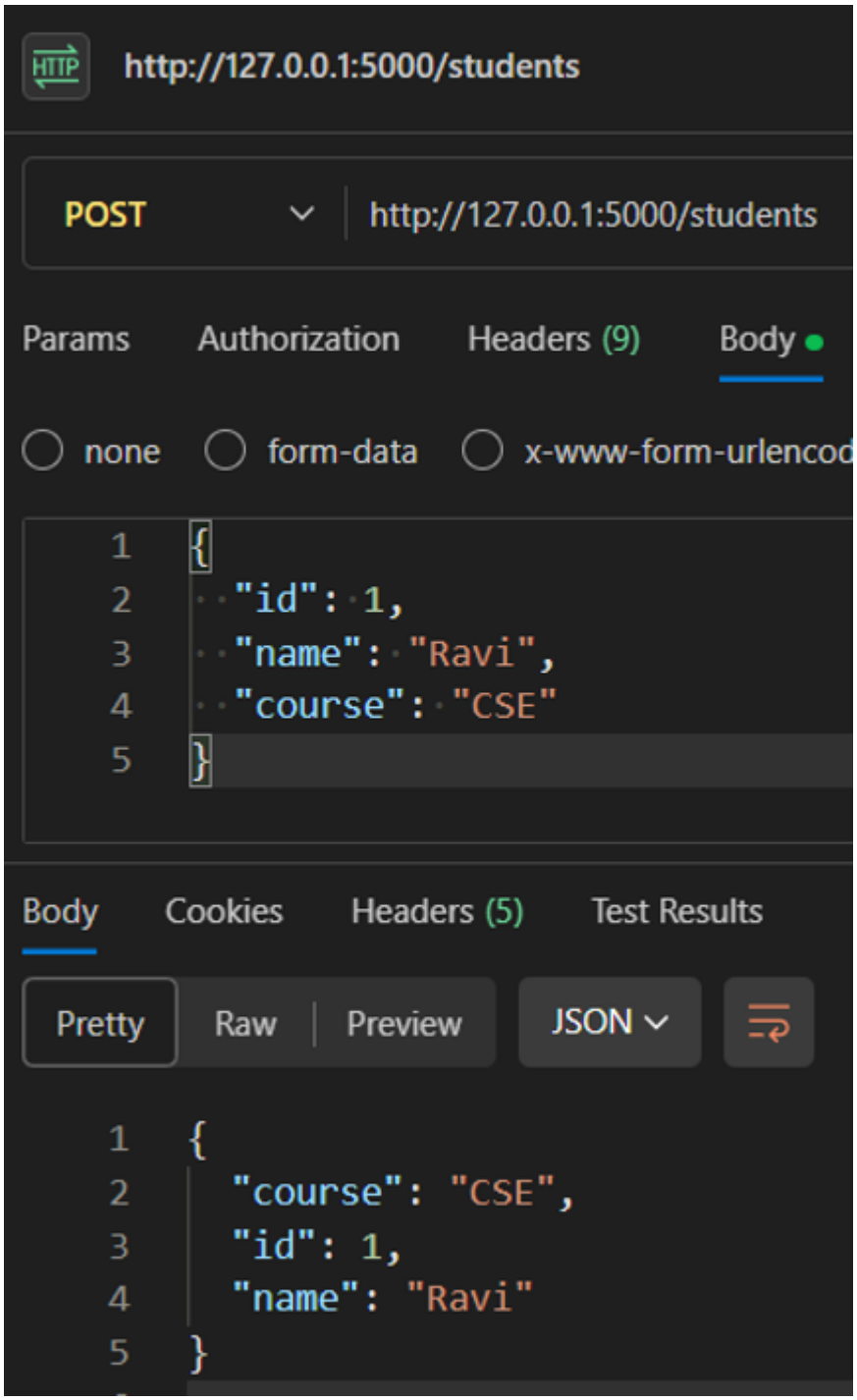
Use in-memory storage and proper validation.

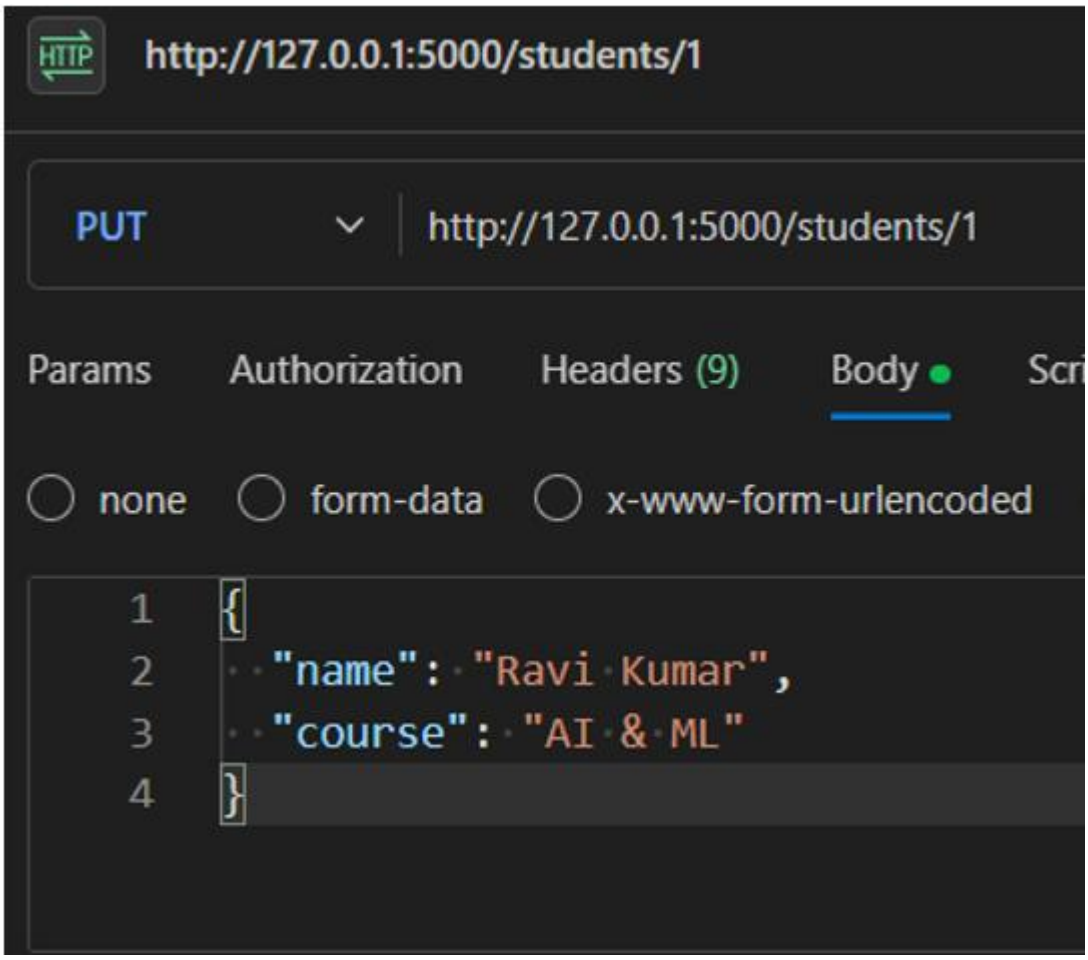
Code:

```
task-03.py > delete_student

Use in-memory storage and proper validation.'''
from flask import Flask, request, jsonify

app = Flask(__name__)

students = []


@app.route('/students', methods=['GET'])
def get_students():
    """Fetch all students."""
    return jsonify(students)


@app.route('/students', methods=['POST'])
def add_student():
    """Add new student."""
    data = request.json

    student = {
        "name": data["name"],
        "rollNumber": data["rollNumber"],
        "course": data["course"],
        "year": data["year"]
    }

    students.append(student)
    return jsonify(student), 201


@app.route('/students/<int:roll>', methods=['PUT'])
def update_student(roll):
    """Update student info."""
    for s in students:
        if s["rollNumber"] == roll:
            s.update(request.json)
```

```python


@app.route('/students/<int:roll>', methods=['PUT'])
def update_student(roll):
    """Update student info."""
    for s in students:
        if s["rollNumber"] == roll:
            s.update(request.json)
            return jsonify(s)

    return jsonify({"error": "Student not found"}), 404


@app.route('/students/<int:roll>', methods=['DELETE'])
def delete_student(roll):
    """Delete student."""
    for s in students:
        if s["rollNumber"] == roll:
            students.remove(s)
            return jsonify({"message": "Student deleted"})

    return jsonify({"error": "Student not found"}), 404


if __name__ == "__main__":
    app.run(debug=True)
```

Output:


http://127.0.0.1:5000/students
POST
http://127.0.0.1:5000/students
Params
Authorization
Headers (9)
Body
none
form-data
x-www-form-urlencod
{
"id": 1,
"name": "Ravi",
"course": "CSE"
}
Body
Cookies
Headers (5)
Test Results
Pretty
Raw
Preview
JSON
{
"course": "CSE",
"id": 1,
"name": "Ravi"
}



Explanation:

This API digitizes university student records for easy management and retrieval. It performs CRUD operations like adding, fetching, updating, and deleting student data.
Roll number acts as a unique identifier for maintaining record accuracy.
The system demonstrates RESTful design and structured JSON data exchange.

Task-04:

Prompt:

Create a Flask backend that integrates OpenWeatherMap API.

Endpoints:

GET /weather/current/<city>

GET /weather/forecast/<city>

Return temperature, humidity, and condition.

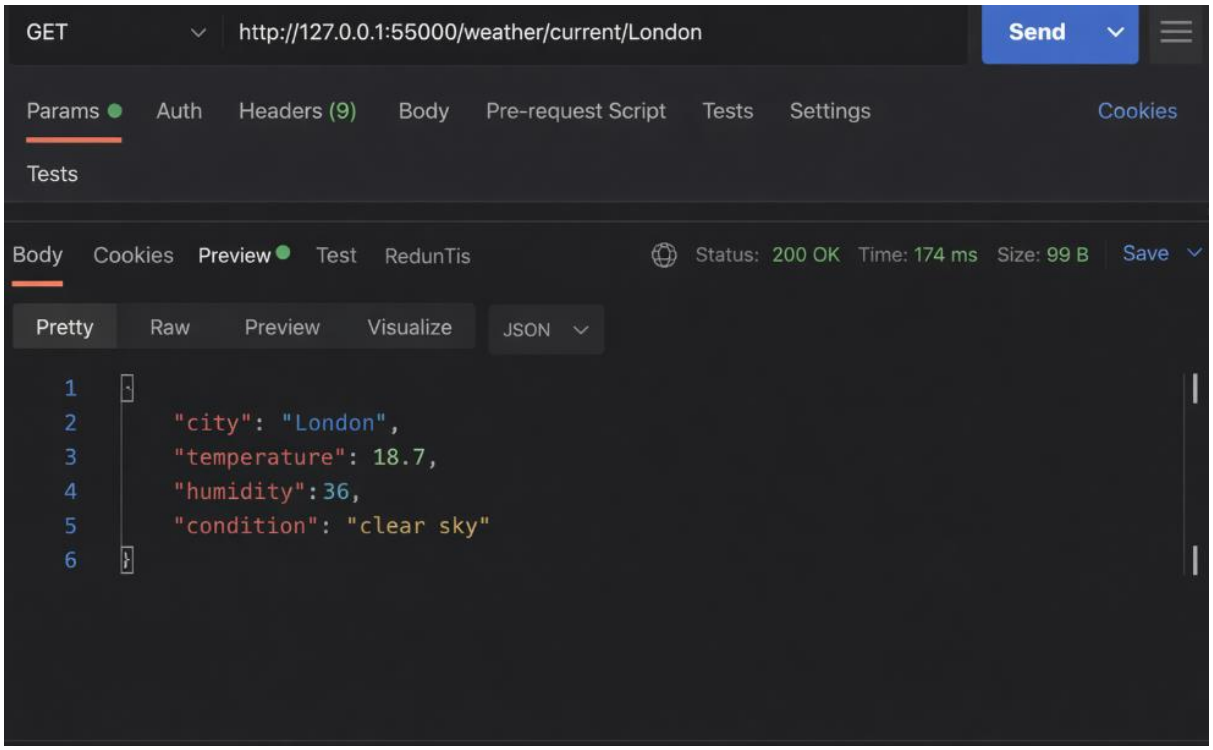
Use requests library.

Code:

task-04.py > forecast

```python
import requests

app = Flask(__name__)

API_KEY = "YOUR_API_KEY"


@app.route('/weather/current/<city>')
def current_weather(city):
    """Fetch current weather."""
    url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"

    response = requests.get(url).json()

    data = {
        "city": city,
        "temperature": response["main"]["temp"],
        "humidity": response["main"]["humidity"],
        "condition": response["weather"][0]["description"]
    }

    return jsonify(data)


@app.route('/weather/forecast/<city>')
def forecast(city):
    """Fetch 5-day forecast."""
    url = f"https://api.openweathermap.org/data/2.5/forecast?q={city}&appid={API_KEY}&units=metric"

    response = requests.get(url).json()

    return jsonify(response["list"][:5])

if __name__ == "__main__":
    app.run(debug=True)
```

Output:



Explanation:

This service integrates a public weather API to provide real-time weather information.
It fetches current weather and forecast data based on a city name request.
The backend acts as a middleware that processes external API responses into clean JSON output.
Users receive temperature, humidity, and weather conditions through simple endpoints.

Task-05:

Prompt:

Create a Flask DELETE endpoint that removes an item from a list by index.

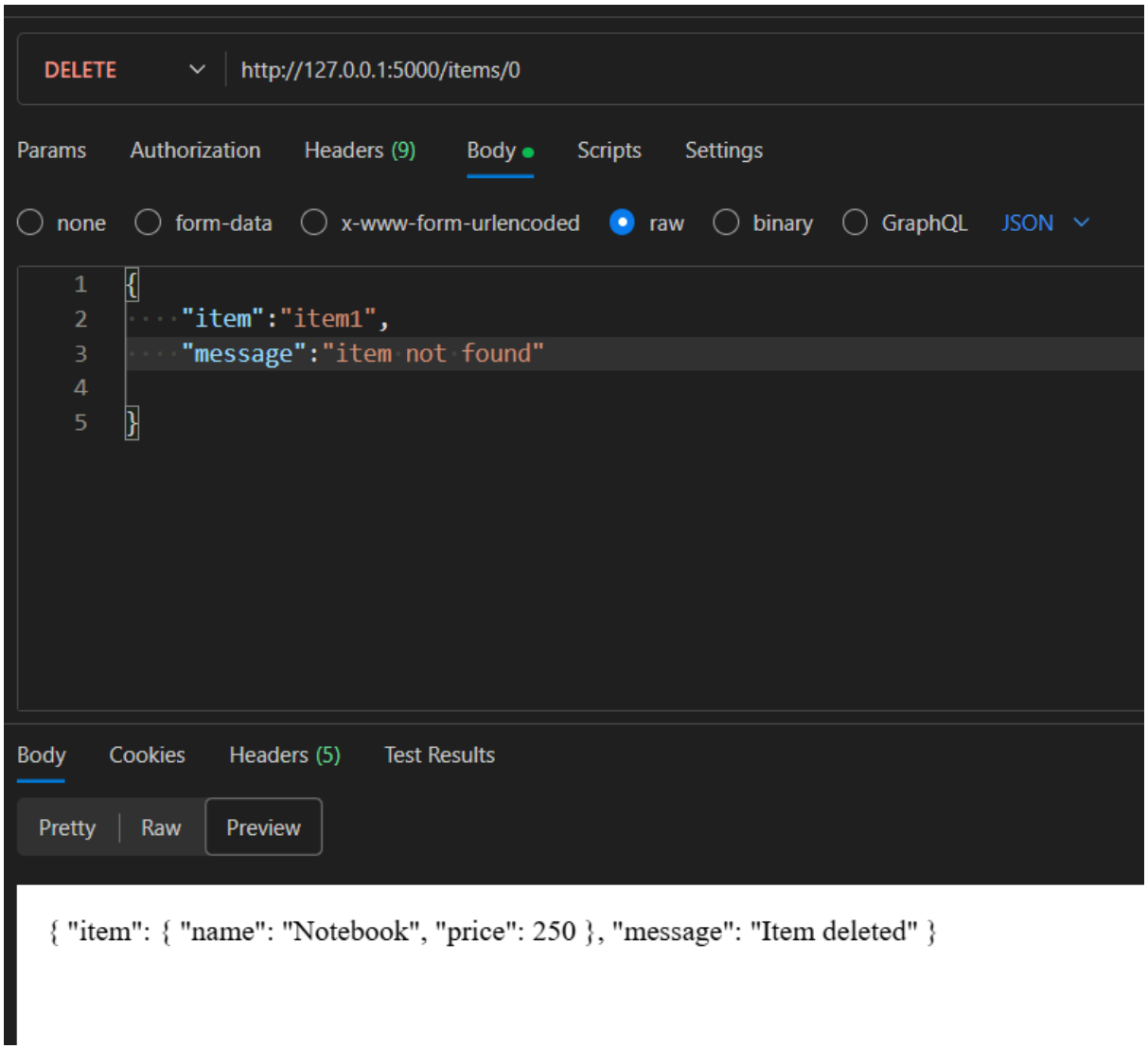
Validate index bounds and return deleted item.

Code:

task-05.py > ...

```python
'''Create a Flask DELETE endpoint that removes an item from a list by index.
Validate index bounds and return deleted item.'''
from flask import Flask, jsonify

app = Flask(__name__)

items = [
    {"name": "Notebook", "price": 250}
]


@app.route('/items', methods=['GET'])
def get_items():
    return jsonify(items)


@app.route('/items/<int:index>', methods=['DELETE'])
def delete_item(index):
    """Delete item by index."""
    if index < 0 or index >= len(items):
        return jsonify({"error": "Item not found"}), 404

    removed_item = items.pop(index)

    return jsonify({
        "message": "Item deleted",
        "item": removed_item
    })


if __name__ == "__main__":
    app.run(debug=True)
```

Output:



Explanation:

This API endpoint removes an item from a collection using its index or ID.
Input validation ensures deletion occurs only when the item exists.
The removed item is returned in the response for confirmation.
It demonstrates safe DELETE operations and proper error handling in REST APIs.